CodeExam Agent Specification

1. Project Overview

CodeExam is a secure web-based coding assessment platform for colleges and organizations.

The platform allows faculty/admins to create coding assessments, students to take assessments individually, execute code in a sandbox, evaluate submissions using public and hidden test cases, monitor active attempts, record student screens, and generate synchronized results, rankings, history, and reports.

This document is the implementation guide for an AI coding agent working on the CodeExam project.

---

2. Technology Stack

Frontend

- React
- Vite
- TypeScript or JavaScript, based on the existing project
- Tailwind CSS
- shadcn/ui where already used
- React Router for routing where applicable

Backend and Database

- Supabase

- Supabase PostgreSQL database

- Supabase Auth for authentication where applicable

- Supabase Realtime for realtime application state where appropriate

- Supabase Storage only when appropriate for application files

Code Execution

- Server-side sandbox execution

- Supported programming languages must run independently from the student's local machine

- Never depend on the student's local Python/compiler installation

- Enforce CPU, memory, execution-time, process, filesystem, and network restrictions

Monitoring

- WebRTC or equivalent realtime streaming technology for live screen monitoring

- Browser MediaRecorder or equivalent for screen recording

- Recording metadata stored in Supabase

- Google Drive may be used for large recording storage when configured

External Storage

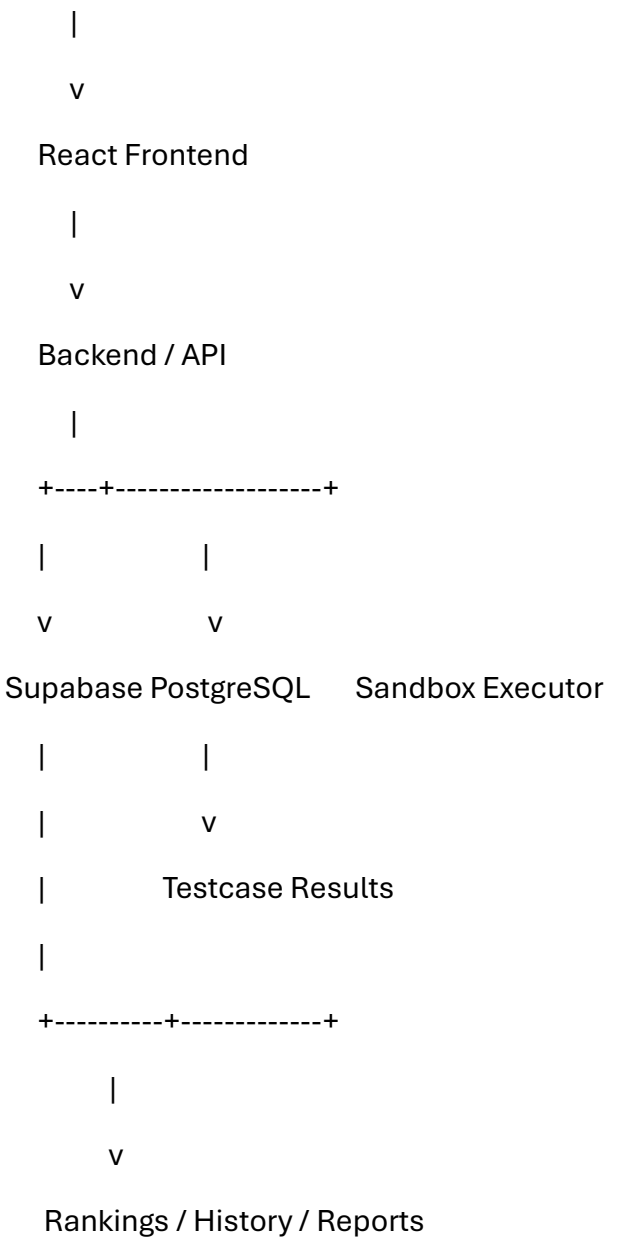
- Google Drive for recordings and other large files where configured

- Supabase stores authoritative metadata and references such as Drive file IDs

---

3. Core Architecture

```text
Student / Faculty / Admin
```


React Frontend
Backend / API
Supabase PostgreSQL
Sandbox Executor
Testcase Results
Rankings / History / Reports
Student Browser
WebRTC
Faculty Live Monitoring
MediaRecorder
Recording Upload

|

v

Google Drive

|

v

Supabase Metadata

```

---

## 4. Non-Negotiable Engineering Rules

1. Do not calculate authoritative marks only on the frontend.

2. Do not trust client-provided scores, ranking values, timer completion, or final assessment status.

3. Supabase/PostgreSQL must be the authoritative source for assessment results.

4. Every student assessment session must have a unique `attempt_id`.

5. Student data must be isolated by authenticated user and attempt.

6. A retake must create a new attempt.

7. Previous attempts must never overwrite or populate a new attempt.

8. Multiple submissions for the same question must not add marks together.

9. Only the latest valid submission for a question contributes to the final question score.

10. A latest submission with zero marks must remain zero.

11. Deleted assessments must not contribute to rankings or aggregate scores.

12. Ended/closed attempts must not remain in live monitoring.

13. Hidden test cases must never be exposed to the student frontend.

14. Code must execute in the sandbox, not on the student's local system.

15. Every critical state transition must be validated server-side.
```

16. Avoid duplicated sources of truth for scores, attempts, settings, and assessment status.

17. Do not use stale localStorage/cache values to overwrite newer Supabase data.

18. Database operations must be safe against duplicate requests and concurrent updates.

19. Existing working functionality must not be broken while fixing another feature.

20. Before changing a database schema, inspect the existing schema and migrations.

---

5. User Roles

Student

Students can:

- View available assessments
- Start an assessment
- Resume an active attempt
- Solve coding questions
- Run code
- Submit code
- View permitted testcase/evaluation results
- Complete the assessment
- View permitted results/history

Students cannot:

- Access hidden test cases

- Modify marks
- Modify rankings
- Access another student's attempt
- Access faculty/admin data
- Modify assessment configuration

Faculty / Examiner

Faculty can:

- Create assessments
- Edit assessments
- Add/edit questions
- Add public and hidden test cases
- Configure marks
- Configure duration
- Configure monitoring
- View active attempts
- Monitor students
- View submissions
- View results
- View rankings
- View history
- View reports
- Close assessments
- Delete assessments where authorized

Admin

Admins can:

- Manage users
- Manage roles
- Manage assessments
- Manage system configuration
- Monitor execution infrastructure
- Monitor recording infrastructure
- Access audit information
- Manage integrations

---

6. Assessment Model

An assessment should support:

- Title
- Description
- Instructions
- Duration
- Start time
- End time
- Maximum marks
- Status
- Allowed programming languages
- Monitoring configuration

- Recording configuration
- Question list
- Assessment version

Recommended statuses:

```text
draft
scheduled
live
paused
closed
deleted
archived
```

Never use multiple inconsistent status fields for the same lifecycle state.

---

7. Attempt Model

Every student assessment session must create:

```text
attempt_id
student_id
assessment_id
```

assessment_version

status

started_at

ended_at

remaining_time

final_score

created_at

updated_at

```

Recommended attempt statuses:

```text

not_started

active

paused

submitted

expired

closed

terminated

```

Attempt isolation

All attempt-dependent data must reference `attempt_id`.

Examples:
```

```text
attempt_id
submission.attempt_id
question_result.attempt_id
recording.attempt_id
monitoring_event.attempt_id
audit_event.attempt_id
```

Do not identify an attempt only using:

```text
student_id + assessment_id
```

because this breaks retakes and historical attempts.

---

8. Retake Requirements

When a student retakes an assessment:

```text
Old Attempt
attempt_id = A001

New Attempt
```

attempt_id = A002

```

A002 must have independent:

- Answers
- Submissions
- Scores
- Question results
- Timer
- Monitoring status
- Recording
- Flags
- Lifecycle status

Never copy old attempt results into the new attempt unless the product explicitly requires it.

---

# 9. Question Requirements

Each coding question should support:

- Question ID
- Assessment ID
- Problem statement
- Input description
```

- Output description
- Constraints
- Examples
- Starter code
- Supported languages
- Maximum marks
- Test cases

Test cases should support:

```text
test_case_id
question_id
visibility
input
expected_output
weight
```

Visibility:

```text
public
hidden
```

Hidden testcase input and expected output must never be returned to the student frontend.

---

10. Testcase-Based Scoring

The score must be calculated from testcase evaluation.

For a question:

```text
total_testcases = N
passed_testcases = P
```

The final question score must follow the configured scoring rule.

Example proportional scoring:

```text
question_score =
  question_max_marks * passed_testcases / total_testcases
```

If weighted test cases are configured, use the configured weights instead.

Important

Do not calculate final marks from the number of submissions.

Incorrect:

```text
submission1 score + submission2 score + submission3 score
```

Correct:

```text
latest_submission(question, attempt).score
```

The authoritative question result should contain:

```text
attempt_id
question_id
latest_submission_id
passed_testcases
total_testcases
score
status
updated_at
```

---

11. Multiple Submission Rule

If a student submits the same question multiple times:

```text
Submission 1 -> 4/10
Submission 2 -> 7/10
Submission 3 -> 10/10
```

Final score:

```text
10/10
```

Do not calculate:

```text
4 + 7 + 10
```

If the latest submission is:

```text
Submission 4 -> 0/10
```

Final score must become:

```text
0/10
```

Do not retain the previous 10/10 score.

Use a deterministic latest-submission rule based on server timestamp and/or monotonically increasing submission sequence.

---

12. Submission Model

A submission should contain:

```text
submission_id
attempt_id
student_id
assessment_id
question_id
language
code/reference
submitted_at
status
passed_testcases
```

total_testcases

score

execution_time

memory_used

error_type

```

Possible statuses:

```text

queued

running

accepted

partial

wrong_answer

compile_error

runtime_error

timeout

memory_limit

security_error

system_error

```

---

13. Code Execution
```

The React frontend must never directly execute untrusted student code on the host machine.

Recommended flow:

```text
React
 |
 | Submit code
 v
Backend
 |
 | Create execution job
 v
Sandbox
 |
 | Compile
 | Execute
 | Run test cases
 v
Evaluation Result
 |
 v
Supabase
 |
 v
React
```

The sandbox must enforce:

- CPU limit
- Memory limit
- Execution timeout
- Process limit
- Filesystem isolation
- Network restrictions
- Temporary workspace isolation
- Cleanup after execution

Compilation/runtime failures must return explicit statuses.

---

14. Timer and Lifecycle

The timer must be based on authoritative server-side state.

Do not trust:

```javascript
Date.now()
```

from the browser as the sole source of truth.

When time expires:

```text
active -> expired
```

Then:

1. Stop accepting normal submissions
2. Finalize latest question results
3. Calculate final score
4. Stop live monitoring
5. Finalize recording
6. Update history/report data
7. Remove the attempt from live monitoring

---

15. Closing the Assessment

If an assessment is closed:

- New attempts must not start
- Existing active attempts must follow the configured close policy
- Live monitoring must not display closed attempts
- The assessment must not remain in an active/live state
- Rankings and reports must update appropriately

If the browser closes or the session terminates, the attempt must not remain permanently marked as live.

Use explicit lifecycle events and heartbeat/timeout mechanisms where necessary.

---

16. Live Monitoring

Faculty live monitoring should display:

- Student
- Registration number where applicable
- Assessment
- Attempt ID
- Attempt status
- Remaining time
- Current question/activity
- Last submission
- Monitoring status
- Recording status
- Relevant flags

Live monitoring should query active attempts from Supabase.

An attempt is not considered live merely because a browser page exists.

Recommended logic:

```text
attempt.status = active
AND
assessment.status = live
AND
heartbeat/session is valid
```

When any required condition becomes invalid, remove it from live monitoring.

---

17. Screen Streaming

Use WebRTC or an equivalent realtime streaming architecture.

Flow:

```text
Student Browser
    |
    | getDisplayMedia()
    v
Screen Capture
    |
    v
WebRTC
```

|

v

Faculty Monitoring

```

Student must explicitly grant browser permission.

If permission is denied:

- Show a clear message
- Do not silently assume monitoring is active
- Apply configured assessment policy

If the stream disconnects:

- Update monitoring state
- Record the event
- Allow recovery where configured

---

# 18. Screen Recording

Use browser screen recording through MediaRecorder or equivalent.

Recommended flow:

```text

Screen Capture

|

v

MediaRecorder

|

v

Chunks

|

v

Upload Service

|

v

Google Drive

|

v

Supabase Recording Metadata

```

Recording metadata should include:

```text

recording_id

attempt_id

student_id

assessment_id

drive_file_id

status

started_at

ended_at

uploaded_at

file_size

error

```

Possible statuses:

```text

starting

recording

uploading

completed

failed

cancelled

```

Never mark a recording as completed before the final upload is confirmed.

---

19. Supabase Database Design

Use PostgreSQL relational integrity.

Recommended core tables:

```text

profiles

assessments

assessment_versions

questions

test_cases

assessment_questions

attempts

submissions

question_results

recordings

monitoring_sessions

monitoring_events

audit_events

```

Optional tables:

```text

classes

faculty_assignments

student_assessments

notifications

system_settings

execution_jobs

execution_results

```

Use foreign keys wherever appropriate.
```

Use indexes for:

```text
attempts.student_id
attempts.assessment_id
attempts.status
submissions.attempt_id
submissions.question_id
question_results.attempt_id
question_results.question_id
recordings.attempt_id
```

---

20. Supabase Row Level Security

RLS is mandatory for sensitive tables.

Students must only read/write records belonging to their own authenticated identity.

Example conceptual rule:

```text
auth.uid() = student_id
```

Faculty must only access authorized assessments.

Admins may access broader data according to role.

Never depend only on frontend route protection.

Frontend authorization is not a security boundary.

---

21. Source of Truth

Use Supabase as the authoritative source for:

- Assessments
- Questions
- Test cases
- Attempts
- Submissions
- Question results
- Final scores
- Rankings
- Assessment settings
- Lifecycle status
- Recording metadata
- Monitoring state
- Audit events

Browser state may be used for temporary UI state but must not overwrite authoritative server state incorrectly.

---

22. Ranking Calculation

Student Rankings must use finalized authoritative attempt/question results.

Conceptually:

```text
obtained_marks =
SUM(final_question_scores)
```

Maximum marks:

```text
max_marks =
assessment.maximum_marks
```

Do not calculate maximum marks from unrelated tests.

Deleted assessments must not contribute to ranking totals.

If multiple attempts exist, apply the configured ranking policy explicitly.

Do not silently combine scores from multiple attempts.

---

23. History and Reports

History and reports must use the same authoritative result data as rankings.

Required synchronization:

```text
Assessment
  |
  v
Attempt
  |
  v
Question Results
  |
  v
Final Score
  |
  +----> Student Result
  +----> History
  +----> Ranking
  +----> Reports
```

Never maintain separate manually calculated marks for each page.

---

24. Deleting an Assessment

When an assessment is deleted:

- It must no longer be available for new attempts
- It must not appear in live monitoring
- Its scores must not contribute to ranking calculations
- Its aggregate result contribution must be removed or invalidated
- Historical records should follow the configured retention policy
- Related recordings should not become orphaned
- Audit records should remain where required

Prefer soft deletion for auditable systems:

```text
deleted_at
deleted_by
status = deleted
```

Do not physically delete important historical records unless explicitly required.

---

25. Settings Synchronization

Assessment settings must have one authoritative source.

Examples:

- Duration
- Maximum marks
- Monitoring enabled
- Recording enabled
- Allowed languages
- Attempt policy
- Scoring configuration

When settings change:

1. Validate the change
2. Persist it in Supabase
3. Update the relevant version/configuration
4. Refresh dependent frontend views
5. Prevent stale client state from overwriting the new value

For an assessment already in progress, use a clear versioning policy.

---

26. React Frontend Architecture

Suggested structure:

```text
src/
├── components/
├── pages/
├── layouts/
├── hooks/
├── services/
│   ├── supabase/
│   ├── assessments/
│   ├── attempts/
│   ├── submissions/
│   ├── monitoring/
│   └── recordings/
├── lib/
├── types/
├── utils/
├── contexts/
└── routes/
```

Keep business logic out of large UI components.

Prefer reusable hooks/services such as:

```text
useAssessment()
useAttempt()
useTimer()
useSubmission()
useLiveMonitoring()
useRecording()
useRanking()
```

---

27. React State Rules

Separate:

Server state
- Supabase records
- Attempt status
- Submission status
- Scores
- Assessment settings

UI state
- Selected question
- Modal visibility
- Editor state
- Temporary input

- Loading indicators

Do not store authoritative marks only in React state.

Do not use localStorage as the final source of score or attempt status.

---

28. Realtime Synchronization

Use Supabase Realtime where realtime updates are required.

Examples:

- Submission evaluation status
- Attempt status
- Live monitoring
- Recording status
- Faculty dashboard updates

Realtime events must be idempotent.

A repeated event must not create duplicate records or double-count scores.

---

29. API / Service Rules

All sensitive operations should be handled through secure backend logic.

Examples:

```text
startAttempt()
submitCode()
evaluateSubmission()
finalizeAttempt()
calculateQuestionResult()
calculateFinalScore()
closeAssessment()
deleteAssessment()
createRecordingReference()
finalizeRecording()
```

The client should request these operations, not independently implement authoritative business logic.

---

30. Error Handling

Every critical operation must have:

- Loading state
- Success state

- Failure state
- Retry behavior where safe
- User-readable error message
- Developer/logging information

Never silently catch important errors.

Avoid patterns such as:

```javascript
catch (error) {
  return [];
}
```

when an empty result could be mistaken for valid data.

---

31. Data Consistency Rules

When updating an attempt:

```text
1. Validate attempt
2. Validate student ownership
3. Validate assessment status
4. Validate attempt status
```

5. Perform operation

6. Persist result

7. Recalculate affected authoritative result

8. Update dependent views through database/realtime state

```

Do not update:

```text

Student Result

Ranking

History

Report

```

with unrelated client-side calculations.

They should derive from authoritative data.

---

32. Concurrency and Idempotency

The platform must handle:

- Double-click submission

- Duplicate API requests

- Browser retry
```

- Network retry
- Two faculty actions at the same time
- Concurrent result updates
- Multiple realtime events

Use:

- Unique constraints
- Database transactions where appropriate
- Idempotency keys
- Server timestamps
- Version numbers
- Conditional updates
- Proper PostgreSQL constraints

---

33. Security Requirements

Mandatory:

- Supabase RLS
- Secure authentication
- Role-based authorization
- Hidden testcase protection
- Sandbox isolation
- Server-side score calculation
- Input validation

- Rate limiting where required
- Secure secret management
- No service credentials in React client
- No Google Drive service-account private keys in frontend code
- No hardcoded API secrets

Use environment variables for public configuration and secure server-side secrets.

---

34. Google Drive Integration

Google Drive is external storage, not the authoritative application database.

Store only metadata/reference in Supabase:

```text
drive_file_id
drive_folder_id
recording_id
attempt_id
upload_status
```

The frontend must never contain private service credentials.

All privileged Drive operations must run server-side.

---

35. Testing Requirements

Create automated tests for:

Scoring

- 0/10 test cases
- 5/10 test cases
- 10/10 test cases
- Weighted test cases
- Latest submission
- Latest submission with zero score

Attempts

- First attempt
- Resume
- Refresh
- Browser close
- Expiration
- Manual close
- Retake

Rankings

- Deleted assessment

- Multiple attempts
- Zero-score question
- Maximum marks
- Partial score

Security

- Student accessing another student
- Student accessing hidden testcase
- Faculty unauthorized assessment
- Direct API manipulation

Execution

- Compile error
- Runtime error
- Timeout
- Memory limit
- Wrong answer
- Accepted solution
- Sandbox failure

Recording

- Permission denied
- Stream disconnect
- Upload failure
- Successful upload

- Retry

- Finalization

---

36. Critical Acceptance Criteria

The implementation is not complete unless all of the following work:

- Testcase-based marks are correct.

- Multiple submissions do not add marks together.

- Only the latest submission determines the final question result.

- Latest zero-score submissions remain zero.

- Deleted tests are excluded from rankings.

- Closed/ended tests do not remain in live monitoring.

- Retakes create isolated attempts.

- Refresh does not create duplicate attempts.

- Student data is isolated through Supabase RLS.

- Hidden test cases are never exposed.

- Code executes in a sandbox.

- History, reports, rankings, and student results show consistent marks.

- Assessment settings remain synchronized.

- Recording status accurately reflects the actual upload state.

- Critical failures are logged and visible.

- No production secrets are exposed in the React frontend.

---

37. Agent Workflow

When modifying the CodeExam project, follow this sequence.

Step 1: Inspect

Before changing code:

- Inspect the existing React structure
- Inspect Supabase configuration
- Inspect database schema/migrations
- Inspect authentication flow
- Inspect assessment/attempt models
- Inspect scoring logic
- Inspect ranking/history/report logic
- Inspect sandbox integration
- Inspect monitoring/recording integration

Do not assume the existing implementation matches this document.

Step 2: Identify the Source of Truth

For every bug, identify:

```text
What data is wrong?
Where is it stored?
Who writes it?
```

Who reads it?

Is there another duplicate source?

```

Fix the source of truth instead of patching individual UI pages.

## Step 3: Make the Smallest Safe Change

Prefer targeted changes.

Do not rewrite working modules unnecessarily.

## Step 4: Validate Data Flow

Trace:

```text
Frontend
→ API/service
→ Supabase
→ Backend processing
→ Supabase result
→ Realtime
→ Frontend
```

Step 5: Test Edge Cases
```

Always test the affected lifecycle and failure cases.

Step 6: Check Regression Risk

Verify:

- Student flow
- Faculty flow
- Admin flow
- Rankings
- History
- Reports
- Live monitoring
- Recording
- Retakes
- Assessment deletion

---

38. Coding Agent Rules

When asked to fix a bug:

1. Find the actual root cause.
2. Inspect related database queries and Supabase policies.
3. Check whether the problem is caused by duplicated state.
4. Fix backend/database logic before adding UI workarounds.
5. Keep React components focused on presentation and interaction.

6. Keep scoring logic server-side.

7. Add or update tests for the bug.

8. Preserve existing behavior outside the affected feature.

9. Do not introduce mock data into production logic.

10. Do not silently fall back to stale localStorage when Supabase fails.

11. Do not remove security checks to make a feature work.

12. Do not expose hidden testcases or private credentials.

13. Do not claim a fix is complete without validating the relevant workflow.

---

39. Definition of Done

A feature is complete when:

- The implementation works in the React frontend.
- Supabase data is correct and consistent.
- RLS/security rules are correct.
- Backend validation is implemented.
- Relevant edge cases are handled.
- Existing workflows continue to work.
- Error states are handled.
- Tests are added/updated.
- No duplicate source of truth remains.
- The feature has been tested using realistic student/faculty workflows.

---

40. Final Product Principle

CodeExam must prioritize:

```text
Correctness
  ↓
Data Integrity
  ↓
Security
  ↓
Assessment Isolation
  ↓
Reliable Execution
  ↓
Consistent Reporting
  ↓
User Experience
```

The most important rule is:

> The database and server-side assessment state are authoritative. The React frontend displays and interacts with that state but must never be trusted to define marks, rankings, attempt ownership, hidden testcases, or final assessment status.